

2026年8月14日 21:24

“接下来我们讲的例子和影像识别有关，给机器一张图片，他要去决定说这张图片里面有什么样的东西，前面的课已经给大家讲过怎么做分类这件事。

“如果最后这个影像辨识系统的向量是一万维的，那么这个系统就能辨识出一万个类别”

“我们最后的输出是 y' ，我们希望 y' 与 y_{hat} 的cross entropy越小越好，接下来的问题是，怎么把一张影像当成model输入呢？”

The diagram illustrates the output of a classification model and the calculation of cross entropy loss. On the left, a 100 x 100 pixel image of a kitten is input into a blue rounded rectangle labeled "Model". The model outputs a vector y' , shown as a column vector with values 0.2, 0.7, 0.1, and a vertical ellipsis. The second element, 0.7, is highlighted with a red glow. To the right, the ground truth vector $\hat{y}$ is shown as a column vector with values 0, 1, 0, and a vertical ellipsis, corresponding to the classes "dog", "cat", and "tree". A blue double-headed arrow labeled "Cross entropy" connects the output vector y' and the ground truth vector $\hat{y}$.

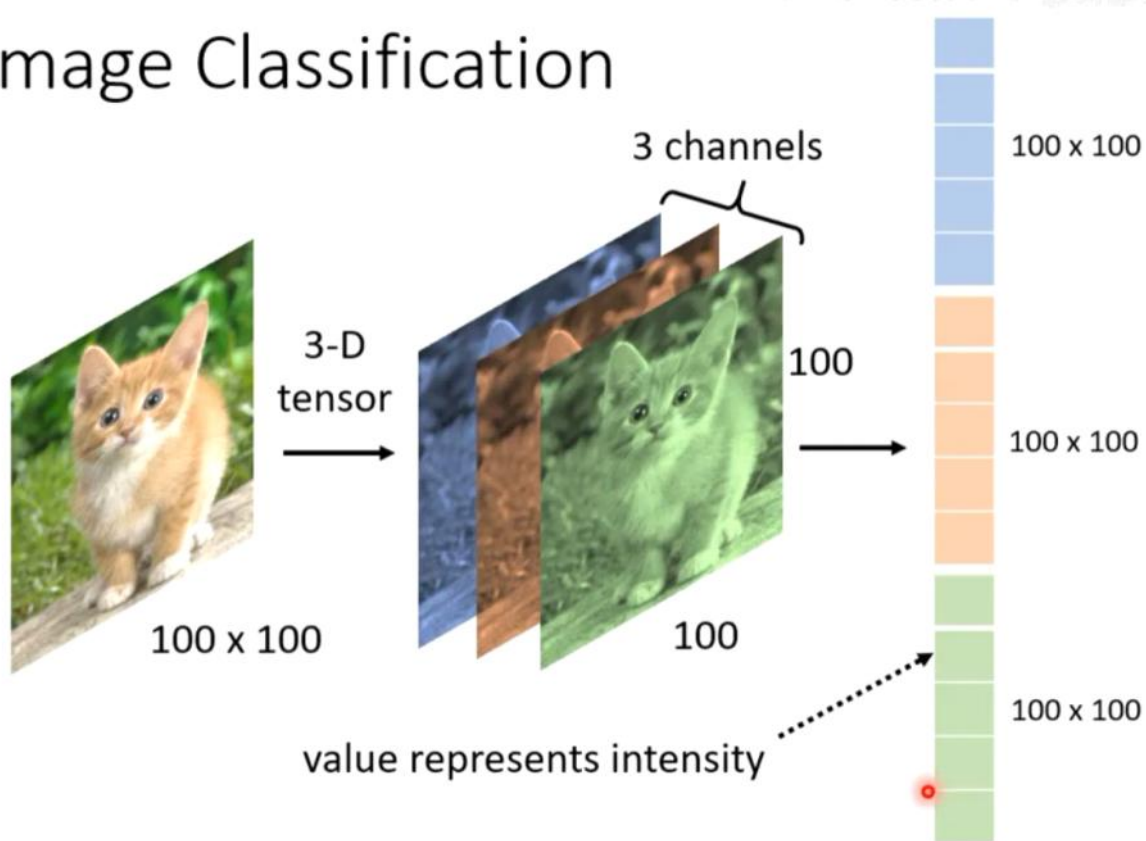
“我们先看一下，一张影像对machine来说是个什么东西？一张图片其实是一个三维的tensor：一维是长，一维是宽，一维是图片channel的数目。”

“一张彩色图片，它的每一个pixel都是由RGB三个颜色组成的，这三个channel就代表了rgb三个颜色，长和宽就代表了这张图片的解析度，我们把一个三维的tensor拉直，拉直以后就可以丢到一个network里去了。知道了吗？到目前为止，我们的输入，它其实都是一个向量。所以我们只要能把一张图片变成一个向量，我们就可以把它当做

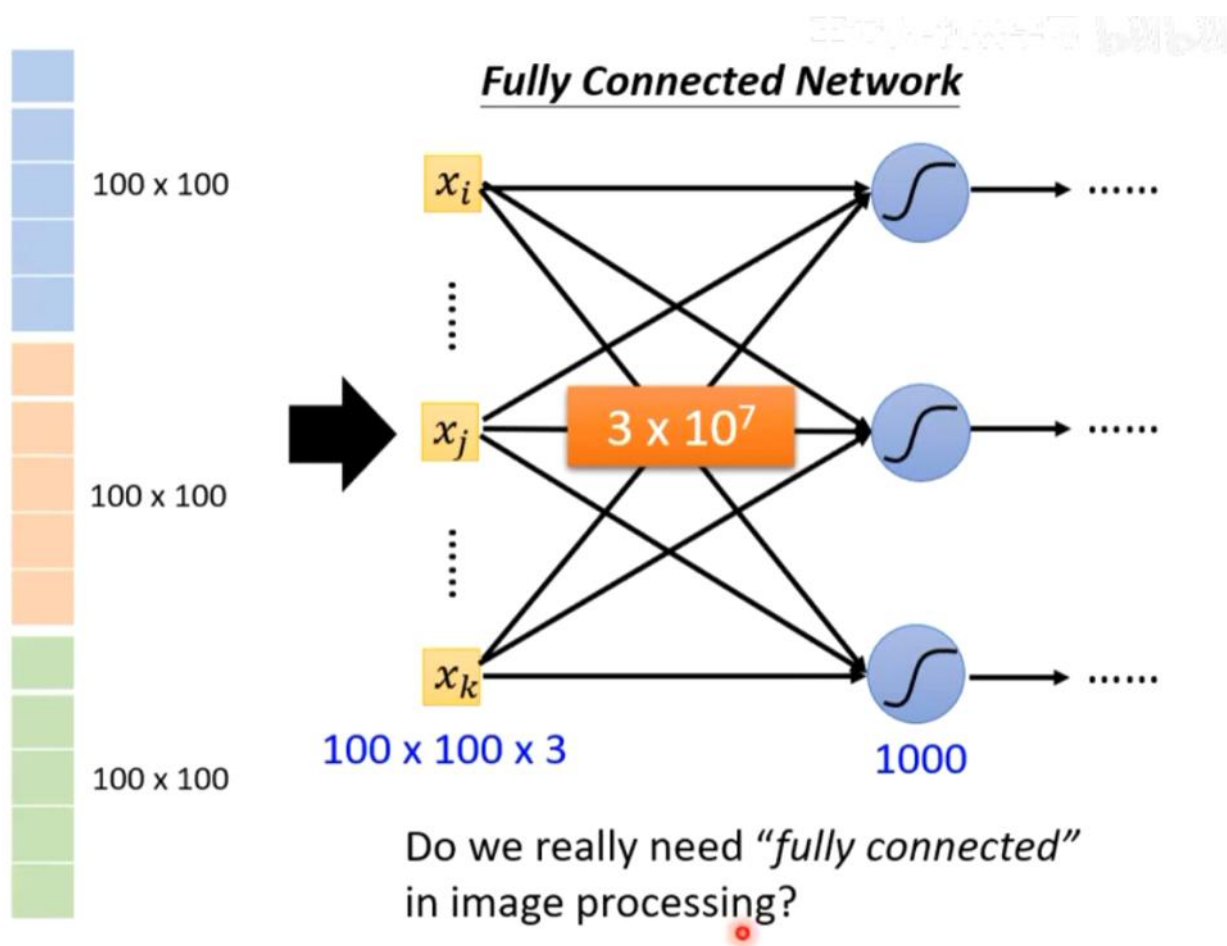
是network的输入，但是怎么把这个三维的tensor变成一个向量呢？最值觉得方法就是拉直他。

“这个向量里每一维中存的数，其实就是某一个pixel，某一个颜色的强度”

Image Classification



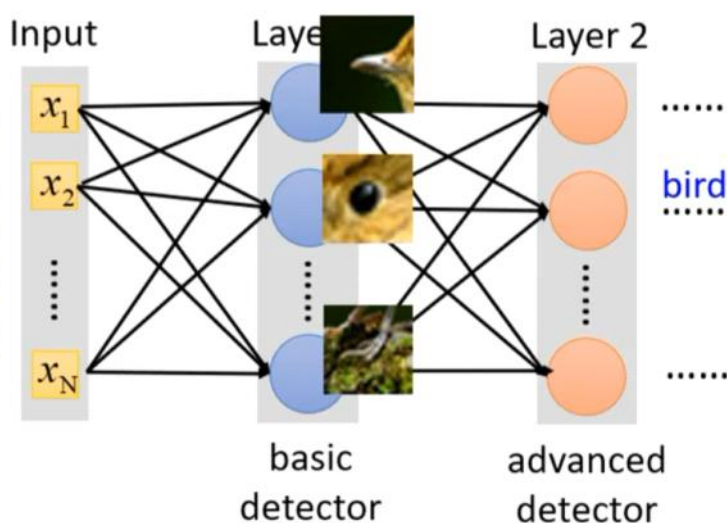
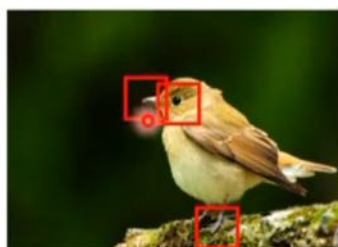
“那么这个向量啊，我们就可以把它当成一个network的输入。可以看到，参数太多了，容易overfitting”



“那么我们如何降参数呢？其实对于network而言，不需要细节的一个一个全看，只需要看到重要的pattern就可以了，比如有的neural看见了鸟嘴，有的看见了眼睛.....感觉就像人一样，第一眼都是先看特点。把这些综合起来，就可以说他看到了一只鸟。（但事实是，人在第一眼看完pattern之后，又回去看其他地方，来补充细节，像这种只看一次的pattern并不好）”

Observation 1

Need to see the whole image?



Some patterns are much smaller than the whole image.



<https://www.dcard.tw/f/funny/p/233833012>

对于图像识别任务，若使用普通的全连接前馈网络，输入是一张 $100 \times 100 \times 3$ 的彩色图片，则第一层每个神经元就要连接 $100 \times 100 \times 3 = 30,000$ 个输入像素，加上偏置共 30,001 个参数。如果网络很宽，参数量会爆炸式增长，导致过拟合风险和计算成本剧增。

“要看这些鸟嘴鸟爪，并不需要看整张图片才需要知道有这些信息。其实只需要看非常小的范围就可以了。所以这些neural并不需要把整张图片当成输入，只需要把图片的一小部分当做输入，就足以让neural侦测到某些特别的pattern有没有出现。——根据这种观察模式，我们就可以做第一个简化 ”

CNN 的第一个简化：只看局部区域

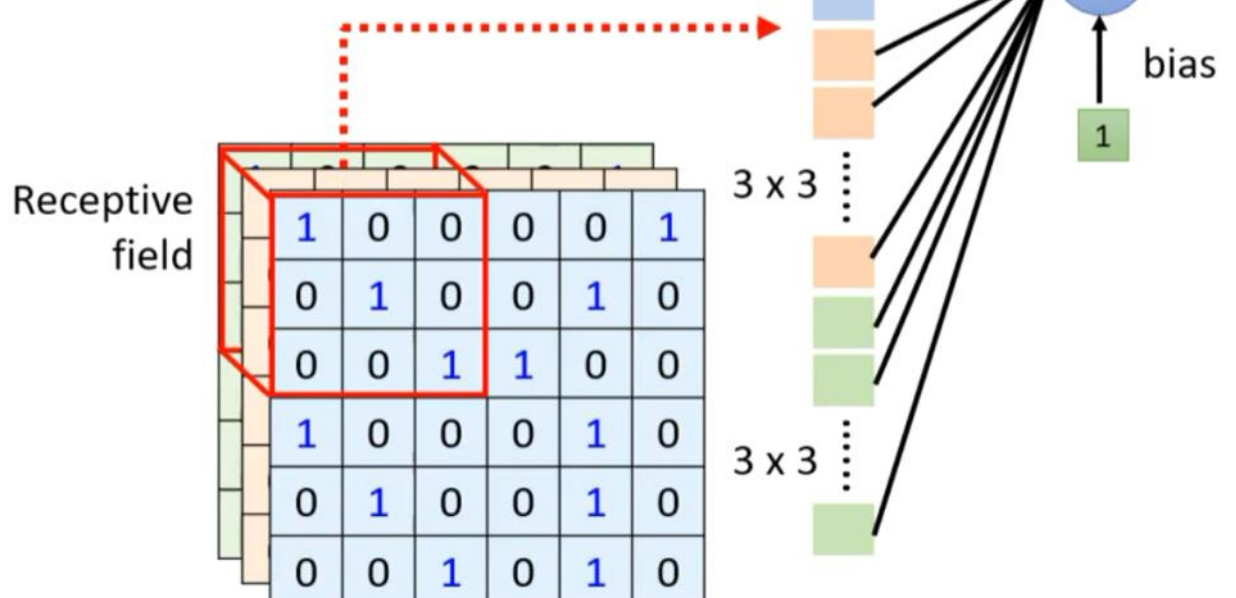
CNN 的第一层神经元不再连接整张图片，而是只看图片中的一个小窗口——这个小窗口就称为「感受野 (Receptive Field) 」。

如 PPT 图所示，每个神经元只关心输入图像中一个 3×3 的小区域，并且同时看这个区域在所有通道（如 RGB 3 个通道）上的值。也就是说，该神经元的输入大小是 $3 \times 3 \times 3 = 27$ 个数值。

每个神经元只需要 $3 \times 3 \times 3 = 27$ 个权重，再加上 1 个偏置 bias，总共只有 28 个参数——相比全连接的 30,001 个参数大幅压缩。

“在CNN里面，有这样一个做法，我们会设定一个区域叫receptive field，每一个neural都只关心自己的receptive field里面发生的事情。举例来说，单独把这个 $3 \times 3 \times 3$ 的receptive field拉直成一个27维的向量作为输入给这个neural”

Simplification 1



感受野设计的灵活性 (PPT 右侧三个问题)

PPT 图中列举了设计感受野时的三个常见疑问，答案是都可以——感受野的具体形状、大小、通道覆盖范围都是超参数，可以根据任务灵活设计。

(1) 不同神经元能否使用不同大小的感受野？

可以。不同神经元可以拥有不同尺寸的 receptive field。例如，一些神经元看 3x3 的小区域捕捉细粒度纹理，另一些神经元看 5x5 或更大的区域捕捉更大范围的结构。这种多尺度感受野在后续网络结构中会自然出现（如 Inception 模块）。

(2) 感受野能否只覆盖部分通道？

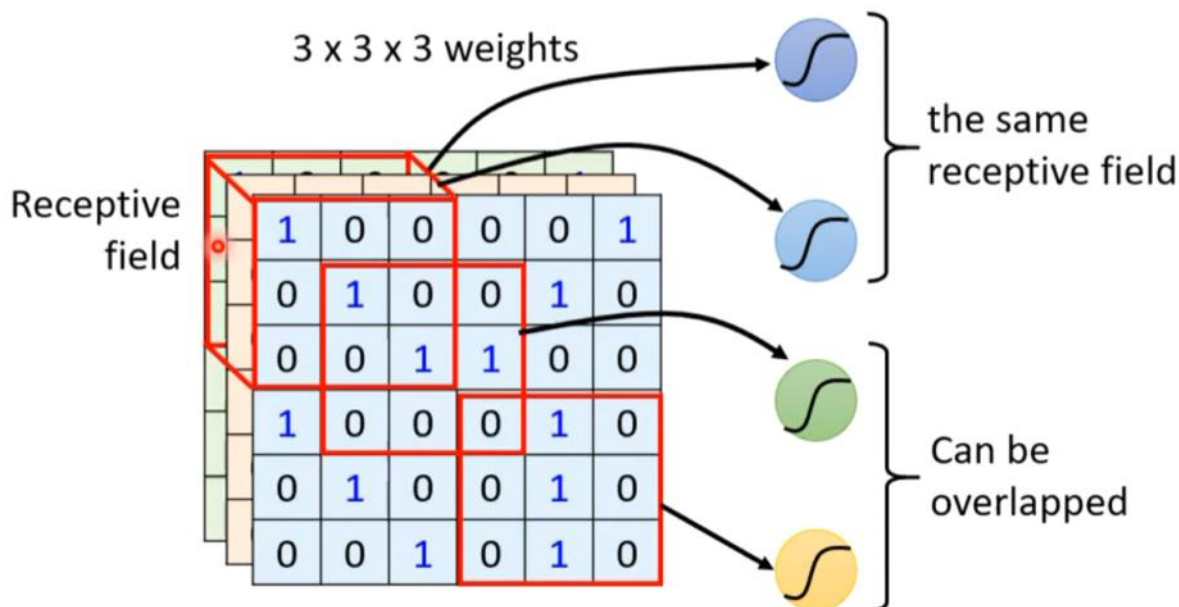
可以。虽然图中示例是 3x3x3（覆盖 RGB 三个通道），但实际并不要求覆盖全部通道。某些神经元可以只看 R 通道或只看 G、B 通道，也可以看任意通道子集。这在多通道特征图（如中间层输出的高维特征）中尤其常见。

(3) 感受野是否必须是正方形？

不必。感受野可以是长方形、长条形或任意形状，取决于任务需求。例如处理文本序列时常见 1xk 的长条卷积核，处理视频时可能是 t x h x w 的三维感受野。正方形只是最常用、最便于实现的默认选择。

Simplification 1

- Can different neurons have different sizes of receptive field?
- Cover only some channels?
- Not square receptive field?



9

典型设置: kernel size / stride / padding / overlap

PPT 这张图展示了 CNN 中感受野的常见配置方式。理解这些术语对后续读论文和调参都很关键。

(1) 每个感受野对应一组神经元

图中每个红框位置并不只产生一个输出值，而是会有一组神经元（例如 64 个）同时观察这同一个局部区域。这 64 个神经元各自拥有独立的权重，可以提取 64 种不同的局部特征。

(2) Kernel size (卷积核大小 / 感受野大小)

图中 kernel size 为 3×3，即每个神经元只看输入中 3×3 的小窗口。3×3 是目前最常用的设置，因为它参数量小、计算快，且多层叠加后感受野足够大。

(3) Stride (步长)

Stride 表示相邻两个感受野中心点之间的距离。图中 stride = 2，意味着窗口每次向右/向下滑动 2 个像素。Stride 越大，输出特征图尺寸越小，计算量越少。

(4) Overlap (重叠)

当 stride 小于 kernel size 时，相邻感受野会相互重叠（如图中 stride=2、kernel=3，窗口之间重叠 1 像素）。适当重叠可以保证相邻区域的局部特征不被遗漏，是 CNN 的常见做法。

(5) Padding (填充)

为了让感受野能覆盖到图像边缘的像素，通常会在图像四周补一圈 0（或其他值），称为 padding。图中右侧边缘就做了填充，使得最右侧一列像素也能被某个感受野的中心或内部覆盖到。

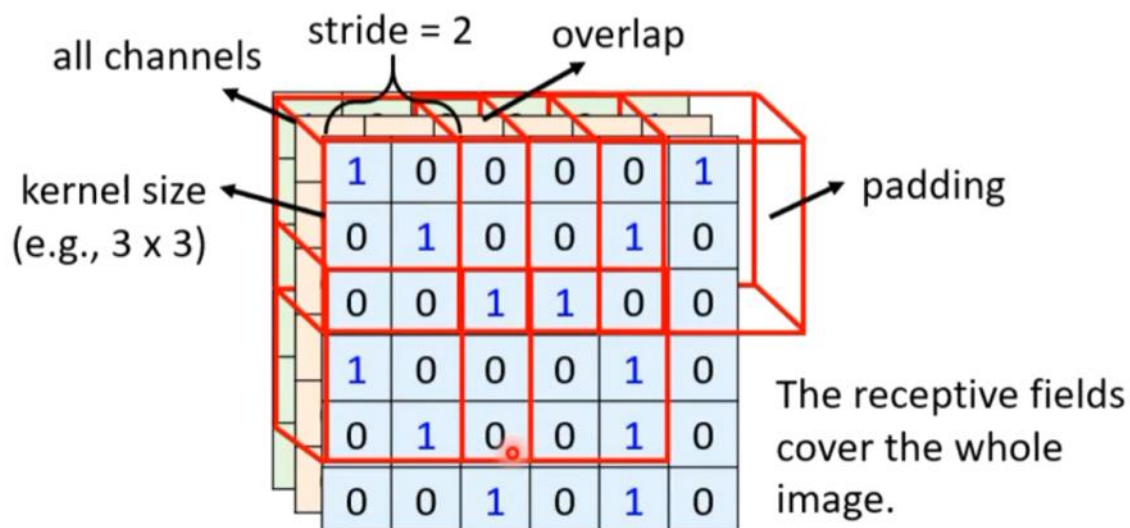
(6) Cover the whole image

通过 kernel size、stride、padding 的配合，所有感受野从左到右、从上到下扫描完整张图片，保证图像中每个位置的信息都能被后续层利用。输出特征图的尺寸由这三个超参数共同决定。

“虽然receptive field有各种各样的方式，但是最经典的是：

Simplification 1 – Typical Setting

Each receptive field has a set of neurons (e.g., 64 neurons).



10

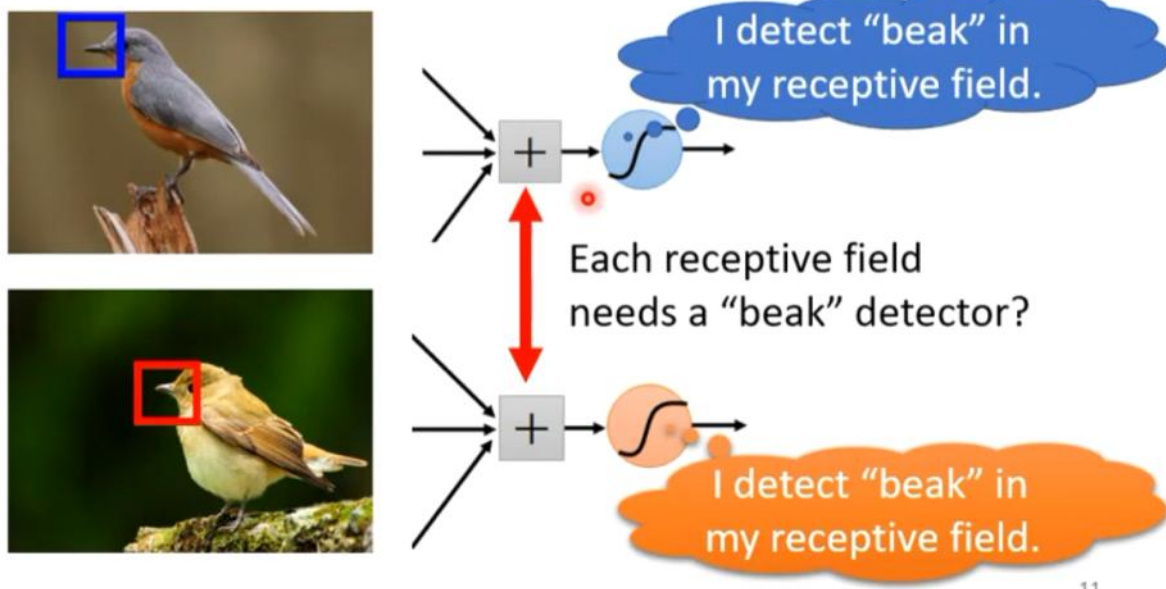
Observation 2: 相同模式出现在不同区域

在图像中，同一个视觉模式（pattern）可能出现在不同的空间位置。例如 PPT 中的鸟嘴（beak）：它可能出现在图片左上角，也可能出现在图片中间或右下角。

核心问题：如果每个 receptive field 都要单独训练一个「鸟嘴检测器」，那我们需要为每个位置都保存一套独立的权重。这不仅浪费参数量，而且学习效率低——同一特征在不同位置被重复学习多次。

Observation 2

- The same patterns appear in different regions.



11

****Simplification 2: 让不同位置共享同一组参数****

CNN 的第二个简化是 ****Weight Sharing (参数共享) ****: 让观察不同位置但负责检测同一种特征的神经元使用完全相同的权重和偏置。

图示解读:

- ① 两个神经元分别位于图像的不同位置, 对应不同的 receptive field (输入分别为 $x_1, x_2, \dots$ 和 $x'_1, x'_2, \dots$)
- ② 它们执行相同的计算:

$$\text{神经元 A: } \sigma(w_1x_1 + w_2x_2 + \dots + b)$$

$$\text{神经元 B: } \sigma(w_1x'_1 + w_2x'_2 + \dots + b)$$

- ③ 关键: 两个神经元共享同一组参数 $w_1, w_2, \dots$ 和同一个偏置 b , 只有输入来自不同位置。

**** 参数共享的本质: 卷积核 / Filter / Kernel ****

共享后的同一组权重就称为一个 ****filter (卷积核 / 滤波器) **** 或 ****kernel****。这个 filter 在整张图片上滑动 (对应不同的 stride), 每到一处就做一次相同的加权求和, 输出一个数值。所有位置的输出组合起来就是一张 ****feature map (特征图) ****。

因此, CNN 第一层的一个 filter 实际上只做一件事: 检测某一种局部特征 (如竖直边缘、鸟嘴纹理)。如果这个 filter 有 $3 \times 3 \times 3 = 27$ 个权重 + 1 个 bias, 那么无论它在多少位置扫描, 参数量都只有 28 个——这就是 weight sharing 带来的巨大压缩。

****参数共享的效果与意义****

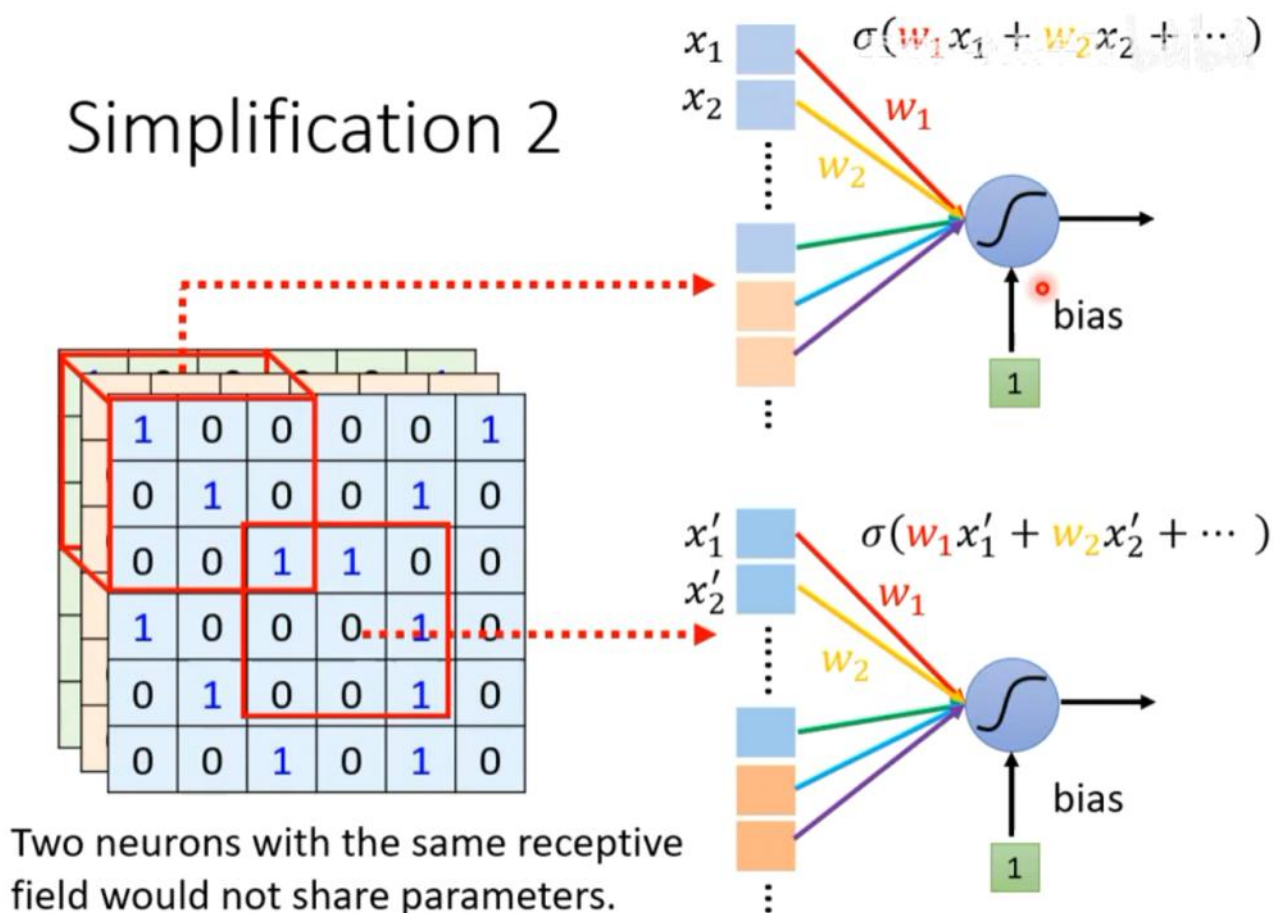
- ① 大幅减少参数量：从「每个位置一套参数」变成「每个特征一套参数」。
- ② 引入平移等变性 (translation equivariance)：同一特征在图像中平移后，输出特征图也相应平移，检测能力不变。
- ③ 提高数据效率：同一组参数在多个位置、多个样本上重复学习，相当于做了数据增强。
- ④ 与 Receptive Field 配合：Receptive Field 解决「看哪里」，Weight Sharing 解决「用什么看」——两者共同构成卷积的核心思想。

这两条观察共同构成了 CNN 的设计动机： ****Some patterns are much smaller than the whole image**** (模式比整张图小 → Receptive Field)； ****The same patterns appear in different regions**** (同一模式出现在不同区域 → Weight Sharing)。

** 应试总结 **

- ① CNN 第二个简化：Weight Sharing (参数共享)， motivated by 「相同模式出现在不同区域」这一观察。
- ② 不同位置的神经元检测同一种特征时共享同一组权重 w 和偏置 b 。
- ③ 共享的权重称为 filter / kernel，扫描整张图后输出 feature map；filter 数量等于输出通道数。
- ④ 效果：参数量剧减、平移等变、数据效率更高。
- ⑤ 卷积层 = 同时加入 Receptive Field + Parameter Sharing 的全连接层特例，对图像任务有更强的归纳偏置 (model bias)。
- ⑥ Receptive Field + Weight Sharing = 卷积 (Convolution) 的核心：局部连接 + 参数共享。

“ 让不同receptive field共享参数？ ——parameter sharing ”



** Typical Setting: filter 数量与共享方式 **

PPT 图展示了 weight sharing 的典型配置：每个 receptive field 有一组神经元（如 64 个），而每个神经元对应一个独立的 filter（filter 1、filter 2、filter 3、filter 4、...）。

关键理解：同一 filter 的权重在所有 receptive field 之间共享。例如 filter 1 的红色权重会在图片的每个 3×3 窗口上使用一次，输出一张代表「filter 1 检测到的特征」的 feature map。如果有 64 个 filter，就会得到 64 张 feature map，即输出通道数为 64。

① filter 数量 = 输出通道数：每个 filter 产生一张 feature map，多个 filter 堆叠成多通道输出。

② 同一 filter 内部参数共享：同一个 filter 的权重在所有空间位置复用。

③ 不同 filter 之间参数独立：filter 1 与 filter 2 学习不同的特征检测器。

Benefit of Convolutional Layer: 卷积层的归纳偏置

PPT 用一张 Venn 图说明了 CNN 与全连接层的关系：卷积层（Convolutional Layer）是同时满足 Receptive Field 和 Parameter Sharing 的网络；而 Receptive Field 和 Parameter Sharing 又都是全连接层（Fully Connected Layer）的特例。

① Fully Connected Layer：「Jack of all trades, master of none」——可以拟合任意函数，没有针对图像任务的先验假设，灵活性最高但容易过拟合。

② Receptive Field：加入「只看局部」的假设，缩小模型搜索空间。

③ Parameter Sharing：加入「同一特征在不同位置复用同一组参数」的假设，进一步缩小搜索空间。

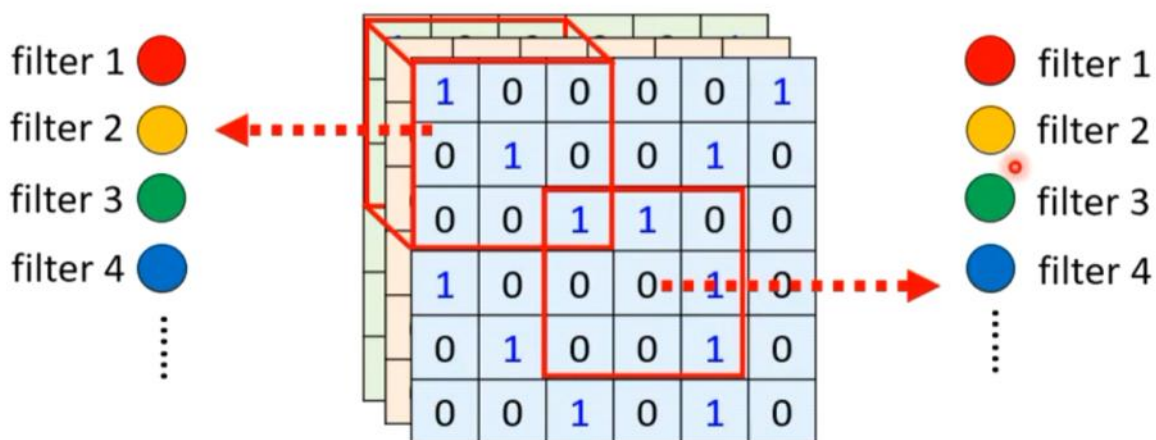
④ Convolutional Layer：同时加入以上两个假设，因此对图像任务有更强的归纳偏置（Larger model bias for image），在图像数据上表现更好。

“ 常见的，在影像辨识上的共享方法？ ”

Simplification 2 – Typical Setting

Each receptive field has a set of neurons (e.g., 64 neurons).

Each receptive field has the neurons with the same set of parameters.

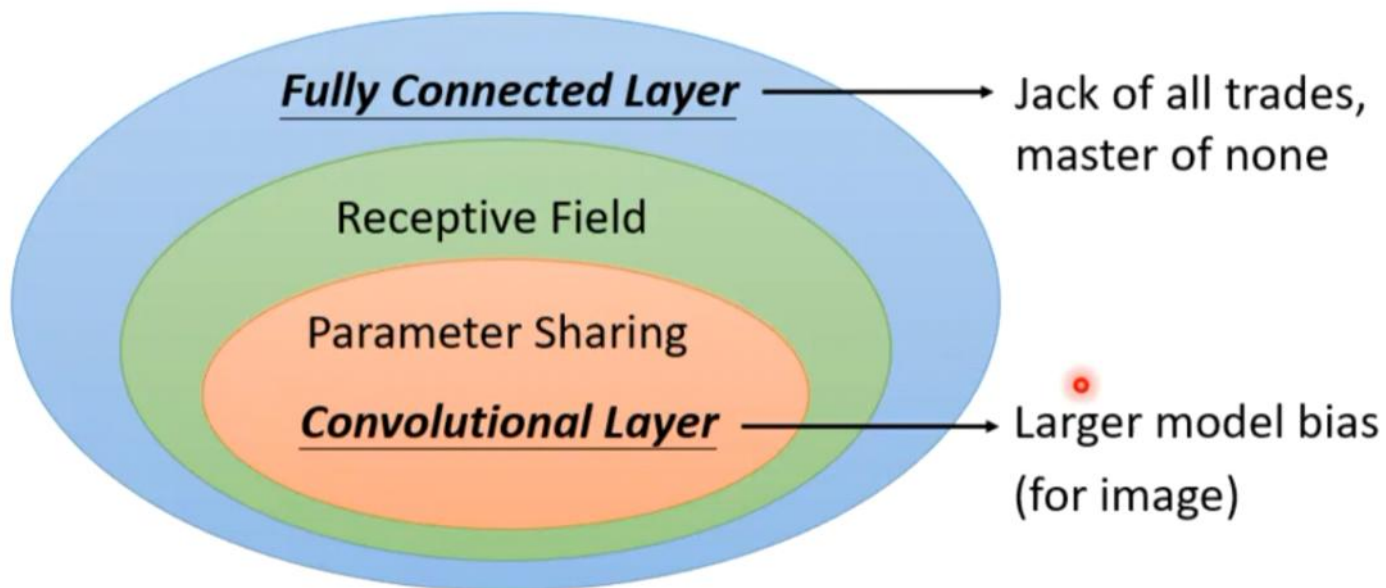


关键特性：局部连接 vs 全连接

- ① 局部连接：每个神经元只连接输入的一小块区域，参数量少，计算效率高。
- ② 捕捉局部特征：图像中的边缘、纹理、角点等低级视觉特征都是局部信息， 3×3 窗口足够表达。
- ③ 平移不变性的基础：同样的局部模式出现在图像不同位置时，可以用同一组参数检测（Simplification 2 会讲 weight sharing）。

“reset field+parameter sharing = convolutional layer”

Benefit of Convolutional Layer



- Some patterns are much smaller than the whole image.
- The same patterns appear in different regions.

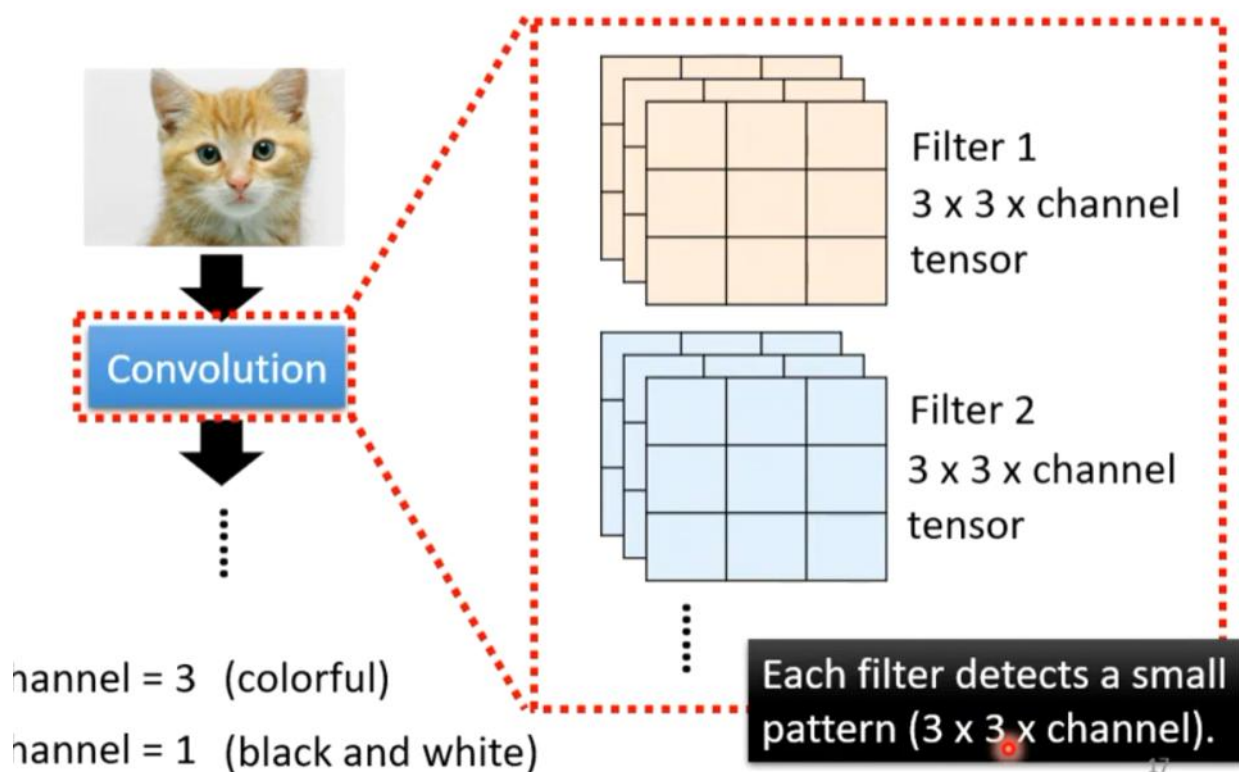
16

Convolutional Layer 的另一种理解：基于 filter

- ① 把卷积层看成「一组可学习的 filter」：每个 filter 是一个 $k \times k \times \text{channel}$ 的权重张量，负责检测某一种局部模式。
- ② 输入 channel = 3 时（彩色图），filter 是 $3 \times 3 \times 3$ ；channel = 1 时（灰度图），filter 退化为 $3 \times 3 \times 1$ 。
- ③ 同一 filter 扫过图像所有位置，输出一张 feature map；filter 扫到哪里，哪里的响应就记录在该 feature map 的对应位置。
- ④ 这种「filter 视角」和之前的「神经元视角」完全等价：每个 filter 就是一组共享的权重，每个位置神经元共享同一 filter。

“第二种讲convolution layer的方法”

Convolutional Layer



“ 这些filter怎么去图片里面抓pattern的呢？我们举一个实际的例子，在接下来的例子里，我们假设channel = 1，也就是说图片是黑白的图片，假设这些filter的参数是已知的，filter就是这一个个的tensor，这个tensor里面的数值我们都是已知的。那么这些filter是怎么把图片上的信息侦测出来的？ ”

****卷积运算示例：6×6 图像 + stride = 1****

① 以第二张图为例：输入为 6×6 单通道图像，filter 大小 3×3，stride = 1。

② 示例 filter（检测竖直边缘）的所有权重为：

第一行：-1 1 -1

第二行：-1 1 -1

第三行：-1 1 -1

中心列权重大、两侧权重小，遇到竖直亮线时输出强正响应。

$$y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x_{i+m,j+n} \cdot w_{m,n} + b$$

④ 将 filter 从图像左上角开始，按 stride = 1 逐步滑动；每到一个位置，做元素相乘再求和（加 bias），得到 feature map 上的一个值。

⑤ 对每一个 filter 重复同样过程；若使用 N 个 filter，就得到 N 张 feature map。

Convolutional Layer

-1	1	-1
-1	1	-1
-1	1	-1

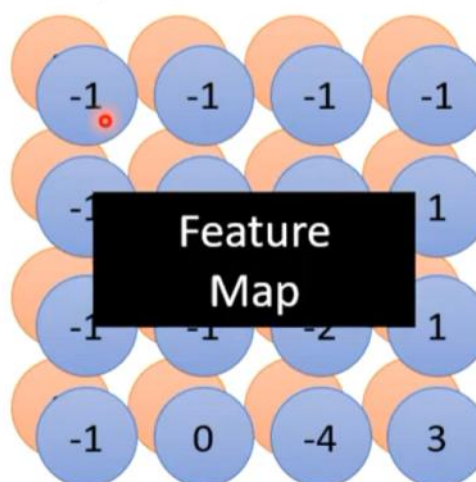
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

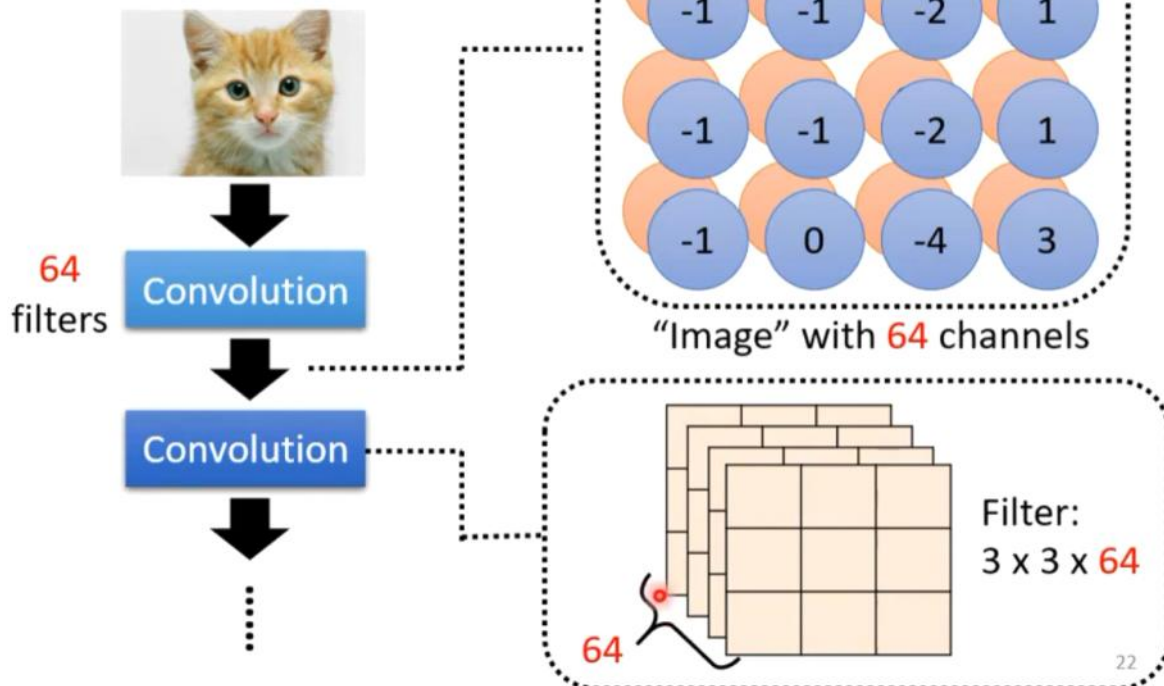
Do the same process for every filter



多层卷积：通道数的堆叠

- ① 第三张图说明：多个卷积层可以串联，前一层的输出作为下一层的输入。
- ② 第一层使用 64 个 filter，输出「64 通道的图像」（严格说是 64 张 feature map 堆叠成的 3D 张量）。
- ③ 第二层的每个 filter 尺寸变为 $3 \times 3 \times 64$ ：每个 filter 同时看 64 个通道，在每个空间位置做 $3 \times 3 \times 64$ 次乘法。
- ④ 通道数 = 前一层 filter 数量；空间尺寸 = 由 stride / padding 决定。
- ⑤ 深层网络通过这种堆叠，逐步从低级特征（边缘、纹理）组合出中级特征（形状、部件）和高级特征（物体、类别）。

Multiple Convolutional Layers



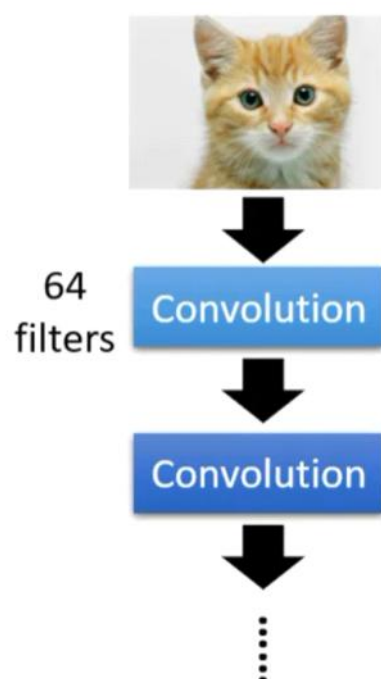
****堆叠卷积的实际效果：有效感受野扩大****

- ① 本图直观说明：虽然每层卷积的 filter 仍是 3×3 ，但两层 3×3 卷积叠在一起，第二层的每个神经元能「看到」原始图像中 5×5 的区域。
- ② 原因：第一层 3×3 filter 输出一个 feature map 点，对应原始图 3×3 ；第二层的 3×3 filter 又覆盖第一层的 3×3 区域，对应原始图 5×5 。
- ③ 有效感受野随网络深度指数级/线性增长（取决于 kernel size 与 stride），使浅层看局部、深层看全局。
- ④ 更深层的 filter 可以组合前层特征，检测更复杂的模式，而不需要一次性使用超大 filter。
- ⑤ 参数量优势：用两个 3×3 卷积代替一个 5×5 卷积，能达到同样大小的有效感受野，但参数更少：

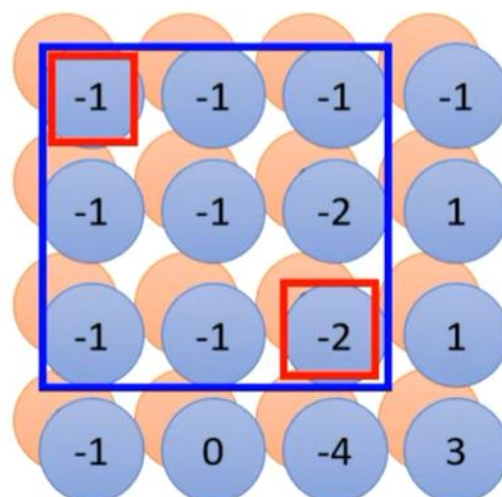
$$2 \times 3^2 = 18 < 25 = 5^2$$

- ⑥ 图中红框/蓝框对应关系：第二层的蓝色大感受野覆盖第一层多个红色小感受野，而第一层每个红色小感受野又对应原始输入的 3×3 区域。

Multiple Convolutional Layers



1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



23

卷积层的两个故事：神经元视角 vs filter 视角**

① 李宏毅课程用两种等价方式讲解卷积层：Neuron Version Story 与 Filter Version Story，本质上是同一个数学操作的不同描述。

② Neuron Version（神经元视角）：

- 每个神经元只连接输入的一个 receptive field，实现局部连接；
- 不同 receptive field 的神经元检测同一种特征时共享同一组权重 w 和偏置 b ，实现参数共享。

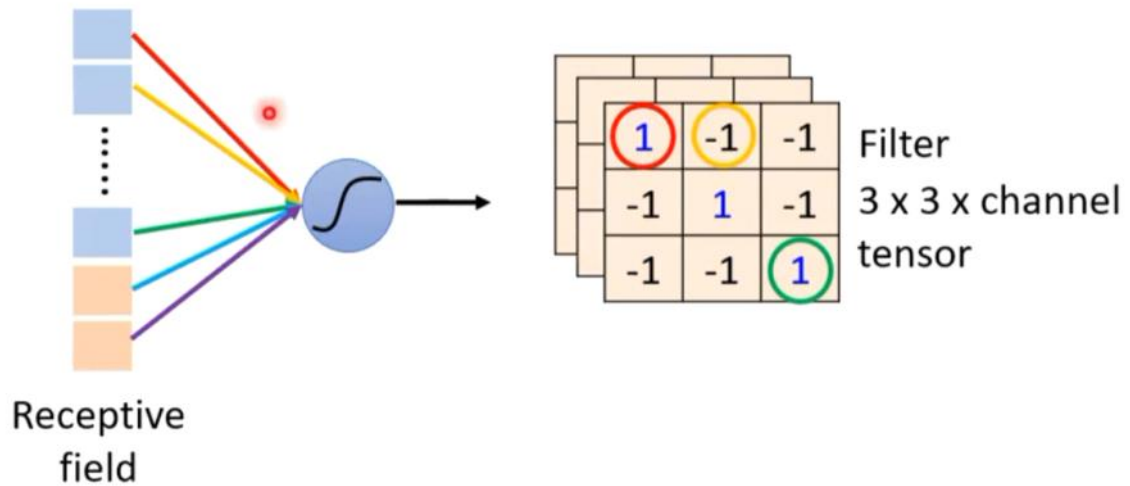
③ Filter Version（filter 视角）：

- 每个 filter 是一个 $k \times k \times \text{channel}$ 的可学习权重张量，负责检测某一种小模式；
- filter 在整张输入图像上滑动做卷积，输出一张 feature map。

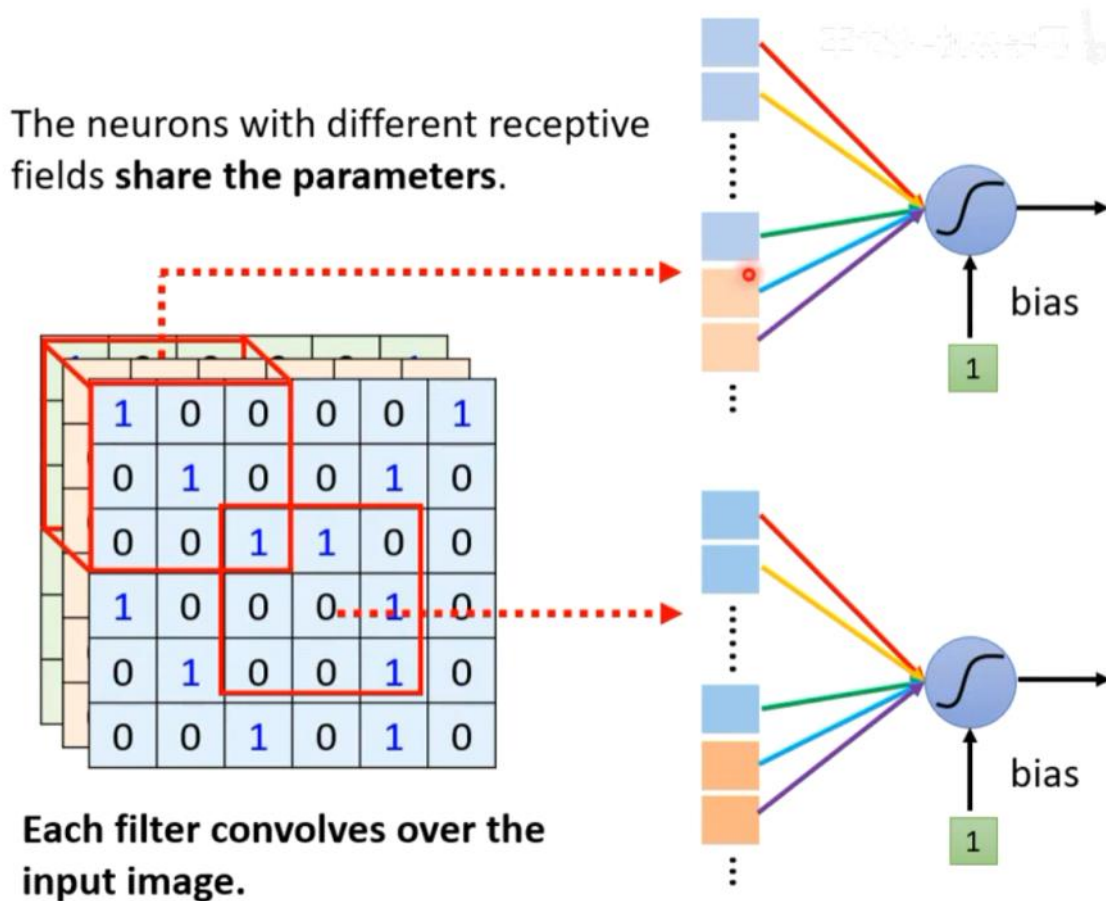
④ 两个视角的数学本质相同：都是在每个空间位置做一组加权求和，再通过激活函数。

$$z = \sigma\left(\sum_i w_i x_i + b\right)$$

Comparison of Two Stories



⑤ 第二张图的形象解读：左下角红框表示不同位置的感受野，箭头指向同一组神经元，说明「不同位置共享参数」；右侧两个神经元的输入连接颜色相同，表示权重——对应相等。



⑥ 第三张图用表格总结如下：

Neuron Version Story	Filter Version Story
每个神经元只看一个 receptive field	一组 filter 检测小模式
不同 receptive field 的神经元共享参数	每个 filter 扫过整张输入图像
They are the same story.	They are the same story.

Convolutional Layer

<u>Neuron Version Story</u>	<u>Filter Version Story</u>
Each neuron only considers a receptive field.	There are a set of filters detecting small patterns.
The neurons with different receptive fields share the parameters.	Each filter convolves over the input image.

They are the same story.

76

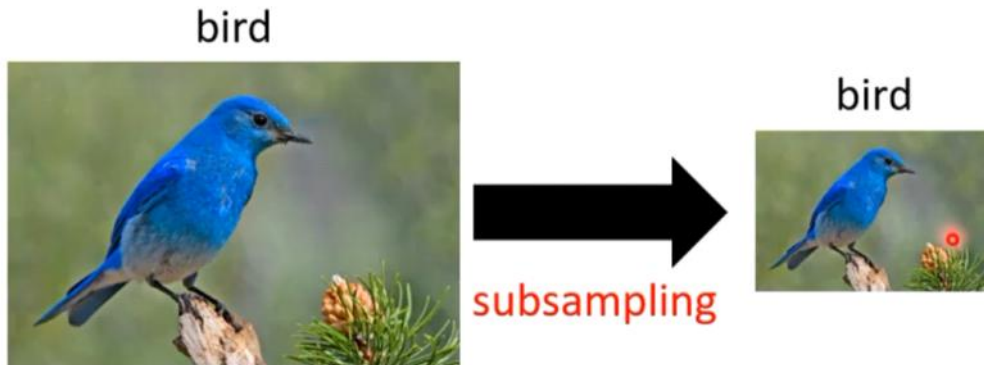
Pooling (池化 / Simplification 3)

****1. Observation 3: 子采样不会破坏物体识别****

- ① 把一张鸟的图片 subsampling (下采样) 后，图像变小、像素减少，但人仍然能认出它是鸟。
- ② 这一观察引出问题：CNN 是否可以直接把 feature map 变小，而不影响识别能力？
- ③ 直接去掉部分像素过于粗暴；更合理的做法是保留局部最显著的信息——这就是 Pooling 的动机。

Observation 3

- Subsampling the pixels will not change the object



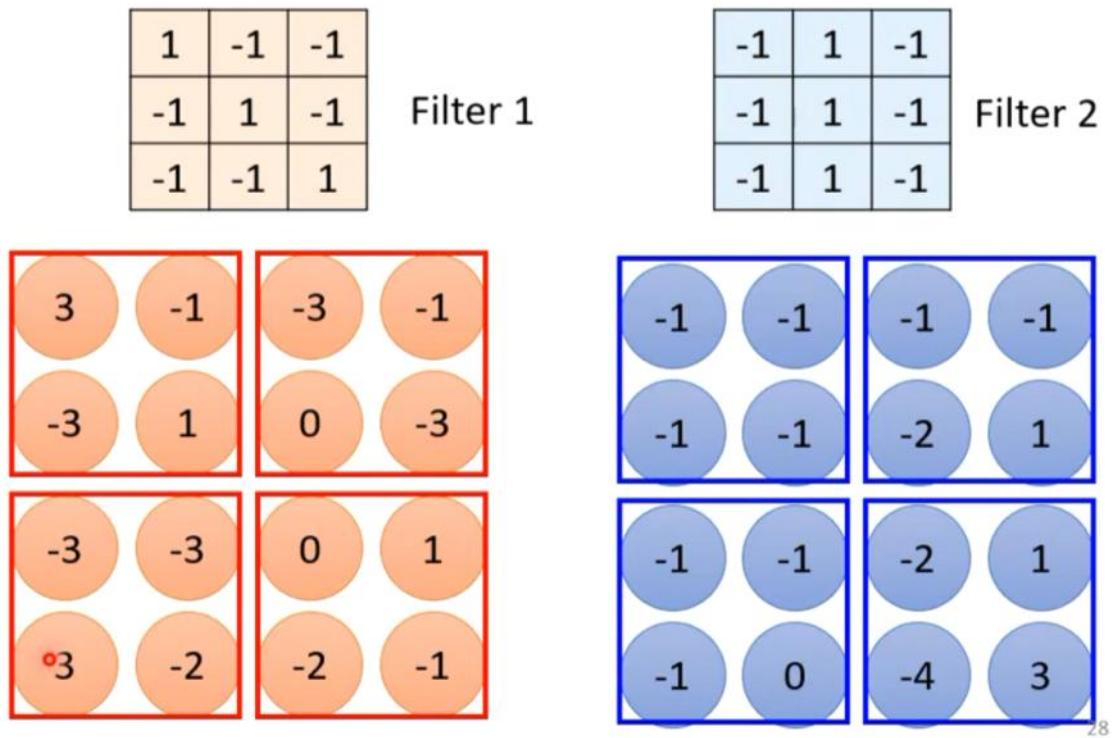
Pooling 的核心机制：以 Max Pooling 为例**

- ① Max Pooling 将 feature map 分成互不重叠的小窗口（常用 2×2 ），每个窗口只保留最大值。
- ② 图示说明：Filter 1 和 Filter 2 各自产生 4×4 feature map，经过 2×2 Max Pooling 后变成 2×2 。
- ③ 窗口内其他值被丢弃，保留的是「该区域内该特征响应最强」的信号。

$$y_{i,j} = \max_{(p,q) \in \mathcal{P}_{i,j}} x_{p,q}$$

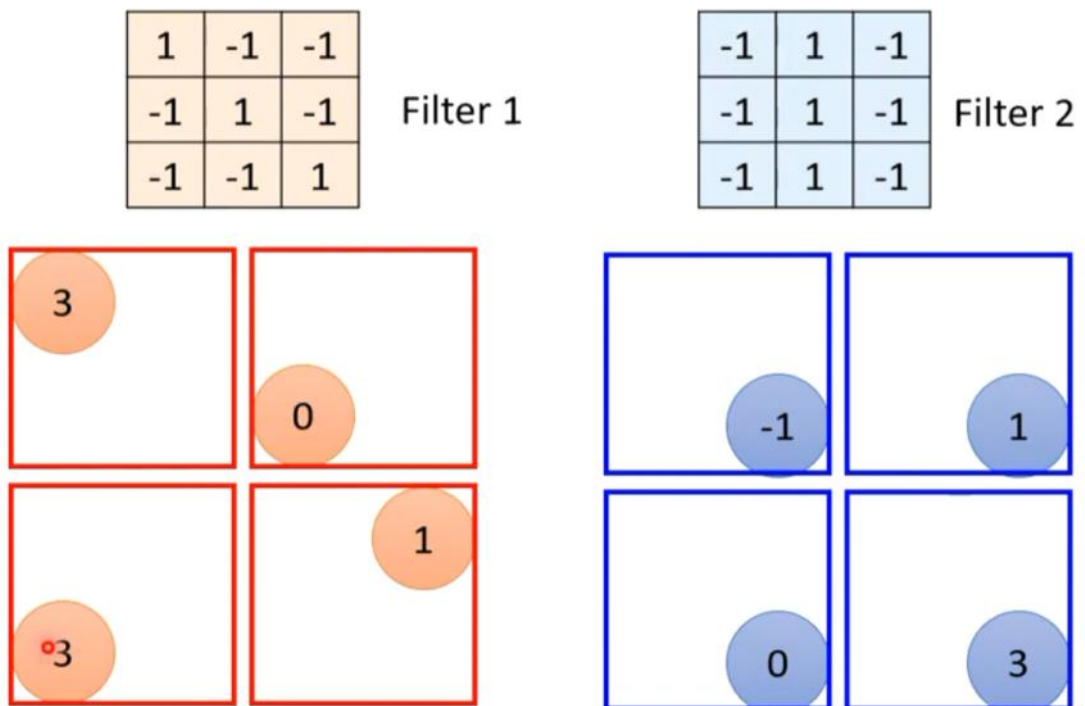
- ⑤ Pooling 没有可学习参数，只执行固定的降采样/聚合规则。
- ⑥ 常见变体：Average Pooling（取平均）、Max Pooling（取最大）；图像任务中 Max Pooling 更常用。

Pooling – Max Pooling



28

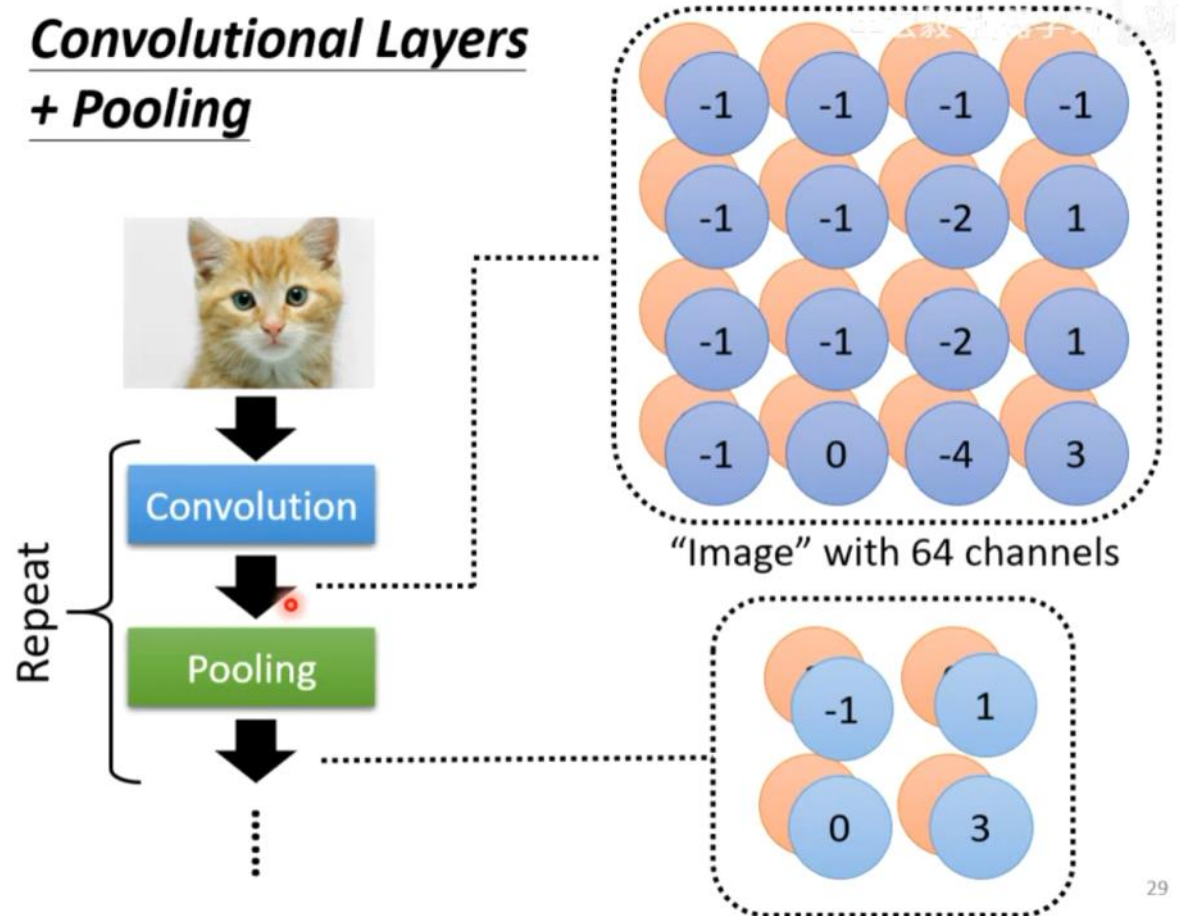
Pooling – Max Pooling



28

Pooling 的效果与意义**

- ① 降低空间分辨率：feature map 高、宽减半 (2×2 pooling + stride=2)，减少后续层计算量。
- ② 扩大后续有效感受野：虽然 feature map 变小，但每个神经元对应的原始图像区域变大。
- ③ 提供一定平移/形变鲁棒性：物体小范围移动后，窗口内的最大值位置可能不变，输出相对稳定。
- ④ 与卷积层分工：卷积负责提取特征，Pooling 负责降采样和抽象。



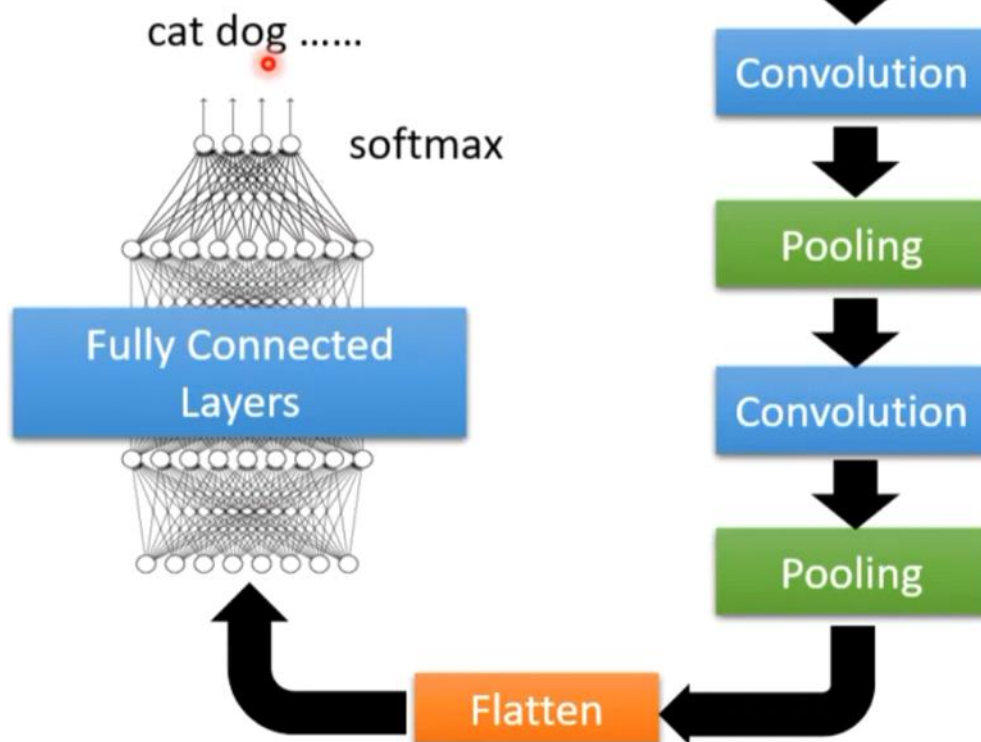
29

“做完几次convolution以后呢，接下来呢，最终怎么得到最后的结果呢？会把output做一件事，叫做 Flatten——把影像里面本来拍成矩阵的样子拉直变成一个向量，再把这个向量丢进fully connected layer里面，最终得到影像辨识的结果。

典型 CNN 架构：Convolution + Pooling 交替重复**

- ① 现代 CNN 的基本积木：Convolution → Activation → Pooling，重复若干次。
- ② 每次卷积增加通道数、提取更丰富特征；每次池化降低空间尺寸、扩大感受野。
- ③ 经过多轮 Convolution + Pooling 后，空间尺寸大幅缩小，通道数大幅增加，最终接全连接层分类。
- ④ 注意：Pooling 层的「图像」不再是原始 RGB，而是高维特征张量（如 64 channels），但结构相同。

The whole CNN

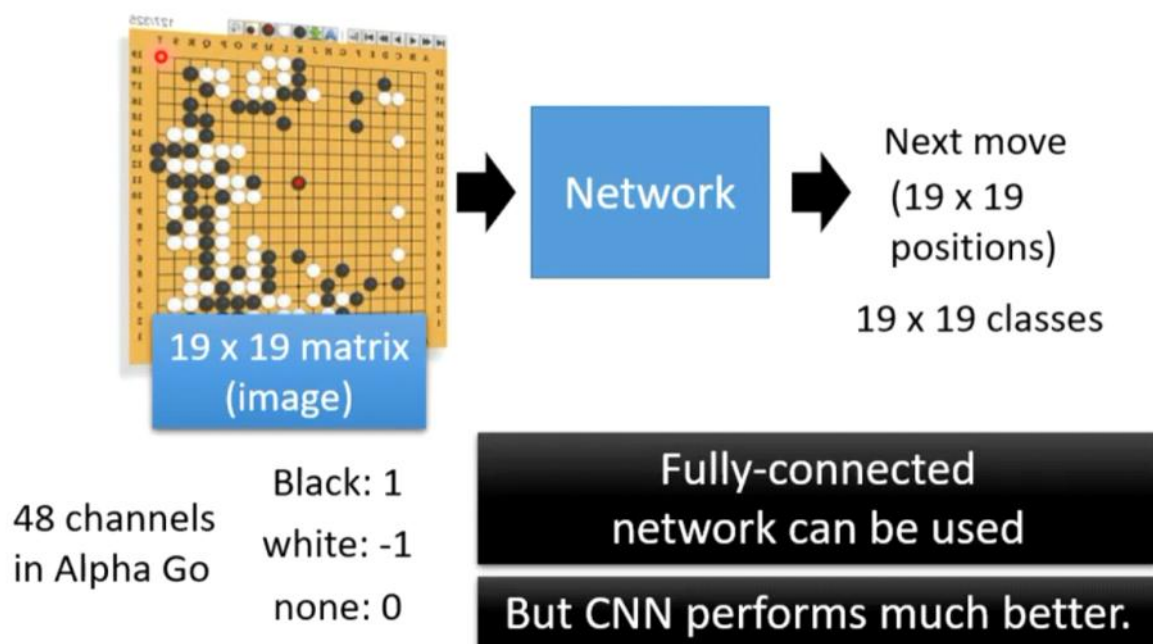


30

“CNN另一个我们耳熟能详的应用是下围棋，其实下围棋就是一个分类的问题，network的输入是棋盘上黑子和白子的位置；输出就是下一步应该落子的位置。可我们今天已经知道network的输入就是一个向量啊，那怎么把棋盘表示成一个向量呢？黑子是1，白子时-1，这是一种表示方法，也有其他表示方法”

“为什么CNN的效果更好呢？其实我们可一把棋盘看成一张19*19解析度的图片，每一个像素每一个pixel就可以看成棋盘落子的位置，那channel呢？一般图像的channel就是RGB，再alpha go里，channels是48？为什么是8？显然是围棋高手设计出来的，比如这个位置是不是要被叫吃了？...”

Application: Playing Go



“为什么CNN可以用在下围棋上呢？并不是说你怎么用都会好的，他是用来做影像辨识的。假如他跟影响没有什么共通的特性的话，你其实不该用CNN的，所以今天既然在下围棋可以用发CNN，那意味着为期跟影响有共同的特性。什么共同的特性呢？”

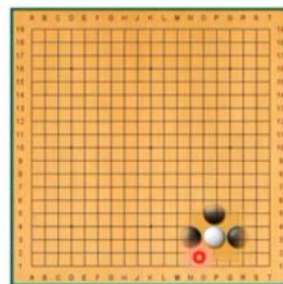
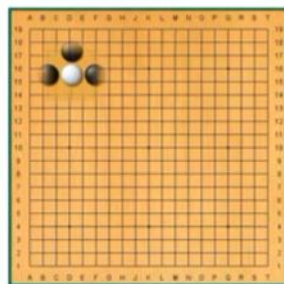
Why CNN for Go playing?

- Some patterns are much smaller than the whole image

Alpha Go uses 5 x 5 for first layer



- The same patterns appear in different regions.



Why CNN for Go playing?

- Subsampling the pixels will not change the object



Pooling

How to explain this???

Neural network architecture. The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Tab. 256 and 384 filters

Alpha Go does not use Pooling

33

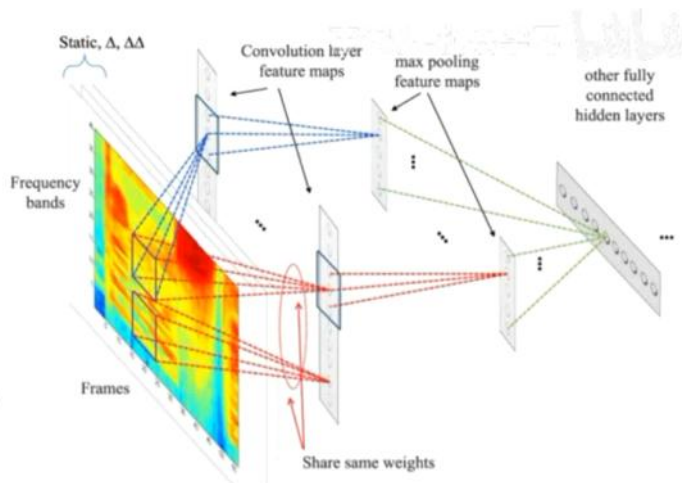
"More applications"

"这个值得看，CNN在语音上的应用"

More Applications

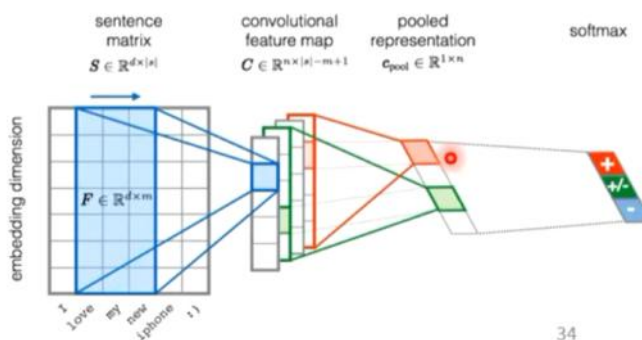
Speech

<https://dl.acm.org/doi/10.1109/TASLP.2014.2339736>



Natural Language Processing

<https://www.aclweb.org/anthology/S15-2079/>



34

“也有人说CNN没办法处理影像放大，旋转的这些事情。旋转拉长之后，向量就变了，filter就认不出来了，也挺容易理解的”

To learn more ...

- CNN is not invariant to scaling and rotation (we need data augmentation 😊).



Spatial Transformer Layer



<https://youtu.be/SoCywZ1hZak>
(in Mandarin)

35

十九、卷积神经网络模型总结 (What is a Convolutional Neural Network)

1. 什么是卷积神经网络模型

①

卷积神经网络 (CNN) 是一类专门处理「网格状结构数据」 (最典型是图像) 的前馈神经网络。

② 从课程视角看, 它本质是带强归纳偏置的全连接网络: 在普通全连接层上额外施加两个结构约束——Receptive Field (局部连接) 与 Weight Sharing (参数共享), 并配合 Pooling (子采样) 构成。

③ 设计动机来自两个观察: 其一, Some patterns are much smaller than the whole image (模式比整张图小) → 只需看局部; 其二, The same patterns appear in different regions (同一模式出现在不同区域) → 应共享参数。再加 Observation 3 (子采样不破坏识别) → 引入池化。

④ 数据流向: 输入是一张三维张量 (高 × 宽 × 通道数, 如 $100 \times 100 \times 3$), 经多层卷积提取特征、多层池化降采样, 最后 Flatten 拉直全连接层, 输出分类/回归结果。

2. 主要功能

① 自动层次化特征提取: 从原始像素自动学习特征——浅层学边缘/纹理, 中层学形状/部件, 深层学物体/语义, 无需手工设计。

② 图像/空间数据的识别与分类: 包括图像分类、目标检测、语义分割、人脸识别、医学影像分析等。

③ 处理具局部相关性与平移重复性的结构化数据: 不仅图像, 还包括棋盘 (围棋)、语音等可被「看作网格」的数据。

④ 降维与抽象: 通过池化逐步降低空间分辨率、扩大感受野, 得到更紧凑的高级表示。

3. 基本原理

① 三大简化 (三个观察 → 三个机制):

· Receptive Field (局部连接): 每个神经元只看一个局部小窗口, 而非整张图 → 参数大幅减少、缓解过拟合。

· Weight

Sharing (参数共享): 检测同一特征的神经元在不同位置共享同一组权重 (filter/kernel) → 参数进一步压缩, 并获得平移等变性。

· Pooling (子采样/池化): 保留局部最显著信息 (如最大值) 而非全部像素 → 降分辨率、扩大感受野、提升鲁棒性。

② 卷积层 = 同时引入 Receptive Field + Parameter Sharing 的全连接层特例, 对图像任务有更强的归纳偏置 (Larger model bias)。卷积运算本质是在每个空间位置做一组加权求和:

$$y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x_{i+m,j+n} \cdot w_{m,n} + b$$

③ 典型架构：Convolution → Activation → Pooling 交替重复，最后 Flatten + 全连接层输出。

****4. 应用情况****

①

计算机视觉：图像分类、目标检测、语义分割、人脸识别、医学影像分析等是最经典落地方向。

② 围棋/博弈：AlphaGo 把 19×19 棋盘看作一张「图片」，每个落子位置是一个 pixel, channel

由围棋先验知识设计（如该位置是否要被叫吃），将「下一步落子」转化为分类问题。

③ 语音处理：语音信号也能组织成类网格结构，CNN 在语音识别中有应用。

④ 其他：视频理解（t×h×w 三维卷积）、文本（1×k 长条卷积）等。

⑤ 前提：数据须与图像具备共同特性（局部相关 + 模式重复出现），否则不该滥用 CNN。

****5. 主要优点****

① 参数效率高：局部连接 + 参数共享使参数量远小于全连接，缓解过拟合、降低计算成本。

② 平移等变性（translation equivariance）：同一特征在图中平移，输出特征图相应平移，检测能力不变。

③ 数据效率高：同一组参数在多个位置、多个样本上复用，相当于隐式数据增强。

④ 层次化特征学习：自动从低级到高级逐级组合特征，免去人工特征工程。

⑤ 归纳偏置强：针对图像任务的更强 model bias，使小数据下也能表现良好。

****6. 存在不足****

① 对几何变换敏感：CNN

难以应对图像的旋转、缩放、平移等变换——图像一旦被旋转或拉长，向量改变，filter 就认不出来（PPT 明确指出）。

② 依赖数据与图像共通的结构特性：若任务数据与图像没有「局部相关 + 模式重复出现」的共性，使用 CNN 反而效果不好。

③ 空间结构在 Flatten 时丢失：接全连接层前需把特征图拉直成向量，破坏二维空间结构；且通常要求输入尺寸统一（需预先缩放）。

④ 可解释性有限：学到的 filter 含义不直观，难以解释「为什么」做出某判断。

⑤ 长程/全局建模弱：感受野与参数共享是人为先验，对全局依赖建模需很深网络才能覆盖大范围，全局关系建模能力不如 Transformer 等结构。

****7. 应试总结****

① CNN 是带强归纳偏置的全连接网络特例：局部连接 + 参数共享 + 池化。

② 三大简化动机：模式比图小（Receptive Field）、同模式跨位置（Weight Sharing）、子采样不破坏识别（Pooling）。

③ 核心公式：卷积（见上图）、最大池化（见下）；卷积层 = 全连接层 + 局部连接 + 参数共享。

$$y_{i,j} = \max_{(p,q) \in \mathcal{P}_{i,j}} x_{p,q}$$

④ 优点：参数少、平移等变、数据高效、层次特征、归纳偏置强；不足：几何变换敏感、依赖网格结构、全局建模弱、可解释性差。